

SOP – Betrieb von CFlux

Standard Operating Procedure (Betriebshandbuch) Version: 1.0 · Stand: 2026-07-13 · Verantwortlich: IT-Betrieb

Diese SOP beschreibt den regulären Betrieb, die Wartung und die Störungsbehebung der CFlux-Plattform (Zeiterfassung / ERP mit Swiss Compliance). Sie richtet sich an Administratoren und den IT-Betrieb.

1. Zweck & Geltungsbereich

- Sicherstellung eines stabilen, sicheren und nachvollziehbaren Betriebs von CFlux.
- Gilt für die produktive Docker-Compose-Umgebung (`docker-compose.yml`).
- Nicht Gegenstand: Fachliche Bedienung der Anwendung → siehe `docs/ADMIN-MANUAL.md`, `docs/USER-MANUAL.md`, `docs/CFLUX-HANDBUCH.md`.

2. Systemüberblick

CFlux besteht aus vier Docker-Containern in einem gemeinsamen Netzwerk (`timetracking-network`):

Dienst	Container	Image / Build	Port (Host)	Zweck
Datenbank	<code>timetracking-db</code>	<code>postgres:16-alpine</code>	<code>127.0.0.1:5432</code>	PostgreSQL 16, Primärdaten
Backend	<code>timetracking-backend</code>	Build aus <code>./backend</code>	<code>3001</code>	Express/Prisma API, Scheduler
PDF-Renderer	<code>timetracking-gutenberg</code>	<code>gutenberg/gutenberg:8</code>	intern	Dokument→PDF-Konvertierung
Frontend	<code>timetracking-frontend</code>	Build aus <code>./frontend</code>	<code>3002</code>	React-App via Nginx (Reverse Proxy auf <code>/api</code>)

Persistente Volumes (Datensicherung hängt an diesen!):

- `postgres_data` → Datenbankinhalt
- `backend_uploads` → hochgeladene Dateien/Anhänge (`/app/uploads`)
- `backend_backups` → automatische & manuelle Backups (`/app/backups`)

Wichtige URLs

- Frontend: `http://localhost:3002`
- Backend-API: `http://localhost:3001`
- Health-Check Backend: `http://localhost:3001/health`
- Public-Settings (Frontend-Healthcheck-Ziel): `http://localhost:3001/api/system-settings/public`

Startsequenz Backend (aus `backend/Dockerfile` → `start.sh`):

1. Warten auf DB, dann `npx prisma db push --accept-data-loss` (Schema-Sync).
2. `node dist/scripts/install.js` (Installations-/Admin-Check, Seed).
3. `node dist/index.js` (API + Scheduler starten).

Beim Start werden im Backend-Prozess zwei Scheduler aktiviert (`backend/src/index.ts`):

- **Backup-Scheduler** (`backupScheduler.service.ts`)
- **Action1-Sync-Scheduler** (`action1Scheduler.service.ts`)

Beide lesen ihre Konfiguration bei jedem Lauf frisch aus den **SystemSettings** in der DB.

3. Rollen & Verantwortlichkeiten

Rolle	Zuständigkeit
IT-Betrieb / Admin	Start/Stop, Updates, Backups, Monitoring, Incidents
Datenbank-Verantwortlicher	Restore-Verifikation, DB-Wartung
Anwendungs-Admin (in CFlux)	Benutzer, Module, Backup-Einstellungen, Action1-Konfiguration
Security-Verantwortlicher	Secrets-Rotation, Sicherheitsupdates, Zugriffskontrolle

4. Betriebsroutinen

4.1 Täglich

- ☐ Container-Status prüfen: `docker-compose ps` — alle Up (healthy).
- ☐ Backend-Health: `curl -fsS http://localhost:3001/health` → 200.
- ☐ Backend-Logs auf Fehler prüfen (siehe §7).
- ☐ Prüfen, dass ein **auto_backup** der letzten Nacht existiert (siehe §6.2).

4.2 Wöchentlich

- ☐ Speicherplatz prüfen: `docker system df` und `Host-df -h`.
- ☐ Backup-Retention kontrollieren (alte `auto_backup_*.zip` werden automatisch bereinigt).
- ☐ Action1-Sync-Status prüfen (Devices/Software aktuell? Admin → Geräte / Software-Report).
- ☐ Log-Volumen/Rotation prüfen.

4.3 Monatlich

- ☐ **Restore-Test** eines Backups in einer Testumgebung (kritisch – siehe §6.4).
- ☐ Sicherheits-/Basisimage-Updates einplanen (Postgres, Gotenberg, Node-Basis).
- ☐ Secrets-Review (JWT_SECRET, DB-Passwort) – siehe §8.
- ☐ Dokumentation/SOP auf Aktualität prüfen.

5. Start / Stop / Neustart

Alle Befehle im Projekt-Root (`/Users/mpue/Documents/devel/cflux`).

```
# Starten (ohne Rebuild)
docker-compose up -d

# Starten mit Neuaufbau (nach Code-Änderungen)
docker-compose up --build -d

# Status
docker-compose ps

# Live-Logs Backend (auf "Server running on port 3001" warten)
docker-compose logs -f backend

# Sauber stoppen (Container weg, Volumes/Daten bleiben erhalten)
docker-compose down

# Einzelnen Dienst neu starten
docker-compose restart backend
```

⚠ **NIEMALS im Produktivbetrieb:** `docker-compose down -v` Das `-v` löscht die Volumes und damit **alle Daten** (DB, Uploads, Backups).

6. Backup & Restore

6.1 Konzept

- Automatische Backups laufen im Backend-Scheduler, gesteuert über die **SystemSettings** (In-App: Admin → Systemeinstellungen). Relevante Felder: `autoBackupEnabled`, Intervall (`daily/weekly`), Uhrzeit, Aufbewahrung (Retention in Tagen).
- Backups werden im Volume `backend_backups` (`/app/backups`) abgelegt als Paar aus `*.json` (Daten) und `*.zip` (Daten + Uploads).
 - Automatisch: Präfix `auto_backup_...`
 - Manuell (Skript): Präfix `backup_...`
- Retention: Der Scheduler löscht `auto_backup_*.zip` älter als die konfigurierte Aufbewahrungsdauer (`cleanupOldBackups`).

🔍 **Wichtig bei Schema-Änderungen:** Neues Prisma-Modell → in BEIDEN `TABLE_MAP` (Controller **und** Scheduler) sowie in der Restore-Logik ergänzen, sonst wird das Modell nicht mitgesichert. Siehe Memory „Backup `TABLE_MAP` sync“.

6.2 Backups prüfen

```
# Vorhandene Backups im Container auflisten
docker exec timetracking-backend ls -lh /app/backups | tail

# Aktuellstes auto_backup finden
```

```
docker exec timetracking-backend sh -c "ls -t /app/backups/auto_backup_*.zip | head -1"
```

6.3 Manuelles Backup

- **In-App:** Admin → Backup & Restore → „Backup erstellen“.
- **CLI (Skript):**

```
docker exec timetracking-backend npm run backup
```

- **Backup aus dem Container sichern (Off-Host!):**

```
docker cp timetracking-backend:/app/backups ./backups-export-$(date +%F)
```

Empfehlung: Automatische Backups zusätzlich regelmäßig **außerhalb des Hosts** sichern (Off-Site), da das Volume sonst mit dem Host verloren geht.

6.4 Restore

⚠ Restore überschreibt Daten. Vor jedem Restore ein aktuelles Backup ziehen. Restore **immer zuerst in einer Testumgebung** verifizieren.

1. **In-App:** Admin → Backup & Restore → Backup-ZIP hochladen → „Wiederherstellen“. (Nginx erlaubt Uploads bis 100 MB; bei größeren Backups `client_max_body_size` in `frontend/nginx.conf` erhöhen.)
2. Nach Restore: Anmeldung, Stichproben der Kernmodule (Benutzer, Zeiteinträge, Rechnungen), Health-Check.

6.5 Roh-Datenbank-Dump (zusätzliche Sicherung)

```
# Dump erstellen
docker exec timetracking-db pg_dump -U timetracking -d timetracking -F c \
  -f /tmp/cflux_$(date +%F).dump
docker cp timetracking-db:/tmp/cflux_$(date +%F).dump ./

# Restore aus Dump (Zielsystem)
docker cp cflux_JJJJ-MM-TT.dump timetracking-db:/tmp/restore.dump
docker exec timetracking-db pg_restore -U timetracking -d timetracking --
  clean /tmp/restore.dump
```

7. Monitoring & Logs

```
# Health
curl -fsS http://localhost:3001/health

# Logs (letzte 200 Zeilen, folgend)
docker-compose logs --tail=200 -f backend
docker-compose logs --tail=200 db

# Ressourcen
docker stats --no-stream
docker system df
```

Beobachtungspunkte in den Backend-Logs:

- 🕒 [Auto-Backup] Next backup scheduled at ... → Backup-Scheduler aktiv.
- 🕒 [Action1-Sync] Scheduler started → Action1-Sync aktiv.
- Prisma-/DB-Verbindungsfehler, JWT-Fehler, unbehandelte Exceptions.

Healthchecks sind in `docker-compose.yml` definiert (DB `pg_isready`, Backend `/api/system-settings/public`). Ein Dienst in `unhealthy` ist ein Alarmsignal.

8. Sicherheit

- **Secrets in `.env`** (nicht im Repo, steht in `.gitignore`):
 - `JWT_SECRET` – muss in Produktion ein starker, einzigartiger Wert sein.
 - `POSTGRES_PASSWORD` – Default `timetracking123` **vor Produktion ändern**.
 - `NODE_ENV=production` in Produktion setzen.
 - `CORS_ORIGIN`, `REACT_APP_API_URL`, `BACKEND_URL`, `FRONTEND_URL` auf die reale Domain (HTTPS) setzen.
- **Default-Admin** (`admin@timetracking.local` / `admin123`): beim ersten Login Passwort ändern; danach den Default deaktivieren/umbenennen.
- **DB-Port** ist bewusst nur an `127.0.0.1` gebunden – nicht nach außen öffnen.
- **TLS/HTTPS**: In Produktion einen Reverse Proxy (z. B. Nginx/Traefik) mit Zertifikat vor das Frontend setzen. `connect-src` in der CSP (`frontend/nginx.conf`) entsprechend anpassen.
- Secrets-Rotation nach Personalwechsel oder Verdacht auf Kompromittierung.
- Action1-API-Token in den SystemSettings sicher verwahren; Rate-Limit (429) beachten (siehe Memory „Action1 API quirks“).

9. Deployment / Updates

```
# 1. Aktuelles Backup ziehen (Pflicht!)
docker exec timetracking-backend npm run backup

# 2. Code aktualisieren
git pull

# 3. Neu bauen & ausrollen
```

```
docker-compose up --build -d

# 4. Migration/Schema-Sync passiert automatisch beim Backend-Start
# (prisma db push). Logs verfolgen:
docker-compose logs -f backend

# 5. Verifikation
curl -fsS http://localhost:3001/health
```

⚠ Das Backend nutzt beim Start `prisma db push --accept-data-loss`. Bei strukturellen Schema-Änderungen **vorher Backup** – potenziell datenverändernd. Deshalb Updates nach Möglichkeit außerhalb der Kernarbeitszeit.

Rollback: Auf den vorherigen Git-Stand zurück (`git checkout <tag/commit>`), `docker-compose up --build -d`, ggf. Restore des vor dem Update gezogenen Backups.

10. Störungsbehebung (Troubleshooting)

Symptom	Erste Prüfung	Maßnahme
Frontend lädt nicht	<code>docker-compose ps</code> , Frontend Up?	<code>docker-compose restart frontend</code>
Login/„500“ bei API	Backend-Logs, DB <code>healthy</code> ?	DB-Verbindung prüfen, <code>restart backend</code>
Backend <code>unhealthy</code>	<code>/health</code> , DB erreichbar?	DB-Status prüfen, Logs, Neustart
Kein Backup entstanden	<code>autoBackupEnabled</code> ? Scheduler-Log?	In-App-Einstellungen prüfen, Zeit/Intervall
Restore/Upload schlägt fehl (413)	Nginx <code>client_max_body_size</code>	Wert in <code>frontend/nginx.conf</code> erhöhen
Action1-Sync leer/Fehler	Token gültig? 429 Rate-Limit?	Token prüfen, später erneut, Logs
PDF-Export defekt	Gotenberg Up?	<code>docker-compose restart gotenberg</code>
„Kein Platz mehr“	<code>df -h</code> , <code>docker system df</code>	Alte Images/Backups bereinigen

Nützliche Diagnose:

```
docker-compose ps
docker-compose logs --tail=200 backend
docker exec timetracking-db psql -U timetracking -d timetracking -c "\dt" | head
```

11. Notfall / Disaster Recovery

Zielsetzung: Wiederherstellung auf einem sauberen Docker-Host.

1. Repository klonen, gesicherte `.env` einspielen.
2. `docker-compose up --build -d` (leeres System hochfahren).
3. Jüngstes gültiges Backup einspielen (In-App-Restore oder `pg_restore`, §6).
4. `backend_uploads` aus Off-Site-Sicherung zurückspielen, falls separat gesichert.
5. Verifikation: Login, Kernmodule, Health-Check.
6. Vorfall dokumentieren; Ursache und Gegenmaßnahmen festhalten.

Voraussetzung für schnelle Recovery: Off-Site-Sicherung von `.env`, DB-Backup und `backend_uploads` (siehe §6.3). Ohne diese ist eine vollständige Wiederherstellung nicht garantiert.

12. Referenzen

- `docker-compose.yml` – Dienste, Ports, Volumes, Healthchecks
 - `backend/Dockerfile` – Startsequenz (`start.sh`), Schema-Sync
 - `backend/src/index.ts` – Scheduler-Start, `/health`
 - `backend/src/services/backupScheduler.service.ts` – Backup-Logik & Retention
 - `backend/src/services/action1Scheduler.service.ts` – Software-/Geräte-Sync
 - `frontend/nginx.conf` – Reverse Proxy, CSP, Upload-Limit
 - `docs/DOCKER.md`, `docs/DEPLOYMENT-FIX.md`, `docs/DATABASE.md`
 - `docs/ADMIN-MANUAL.md`, `docs/CFLUX-HANDBUCH.md`
-

Änderungen an dieser SOP über Git versionieren. Bei jeder Architektur- oder Backup-Änderung diese SOP mit aktualisieren.